Exam Study Guide: Memory Virtualization

Mechanisms of Relocation, Segmentation, and Paging

Prepared for CSE323 Student (North South University)

August 24, 2026

Contents

1 Core Concepts & Definitions

Before solving the mathematical and analytical problems, you must master the fundamental concepts. These are frequently tested as definition or conceptual questions in exams.

The Illusion of Memory Virtualization

The Operating System virtualizes physical memory (RAM) by presenting each running process with the illusion of a private, continuous, and highly secure address space starting at virtual address 0. This is achieved through cooperation between the OS (which manages policies and bookkeeping) and the CPU's Memory Management Unit (MMU), which intercepts and translates addresses on-the-fly.

1.1 Key Goals of Multiprogramming & Virtualization

- **Transparency:** Processes are completely unaware that memory is shared. Each process is written and compiled as if it has sole access to the RAM starting at address 0.
- **Protection (Isolation):** The OS prevents one process from accessing or corrupting the memory of another process or the kernel. If a process attempts an unauthorized access, the hardware catches it, throwing a **Segmentation Fault (SegFault)**.
- **Efficiency:** High space efficiency (minimizing memory waste/fragmentation) and high time efficiency (address translations must be extremely fast, facilitated by hardware like the TLB).

1.2 The Evolutionary Journey of Virtualization

1. **Time-sharing (Bad):** Run one process at a time. When switching, write the entire contents of RAM to disk. This is extremely slow due to disk I/O latency.
2. **Static Relocation (Software-based):** The OS loader rewrites the program's binary code just before execution, adding a physical offset to all instruction pointers. There is no physical/virtual separation, no runtime protection, and processes cannot be moved once loaded.
3. **Dynamic Relocation (Base and Bounds):** A hardware-assisted mechanism. The MMU adds a **Base Register** offset to every virtual address: $PA = VA + Base$. A **Bounds Register** ensures $VA < Bounds$. It prevents memory corruption but requires contiguous allocation, wasting huge physical RAM on the unused gap between the heap and the stack.
4. **Segmentation:** Instead of one base/bounds pair for the entire process, the MMU keeps one base/bounds pair per logical area (Code, Heap, Stack). This allows non-contiguous placement in physical memory but causes **External Fragmentation** (tiny unusable gaps between segments).
5. **Paging:** Breaks memory down into fixed-size blocks (pages and frames). This completely eliminates external fragmentation and allows non-contiguous allocation at a fine-grained level.

1.3 Types of Fragmentation

- **External Fragmentation:** Unused physical memory gaps that exist *between* allocated blocks (e.g., between segments). The total free space may be enough for a process, but because it is scattered in tiny pieces, it cannot be allocated contiguously.
- **Internal Fragmentation:** Unused physical memory that exists *inside* an allocated unit (e.g., inside a page). For instance, if a page size is 4 KB and a process requests 1 byte, the remaining 4095 bytes in that page are wasted because memory is allocated at page granularity.

2 Segmentation Mechanisms

2.1 The Division of Virtual Address

In a segmented system, a Virtual Address (VA) is split into:

$$\text{VA} = [\text{Segment Selector (MSB bits)} \mid \text{Offset (LSB bits)}]$$

- **Segment Selector:** Identifies which segment's Base and Bounds registers to use.
- **Offset:** Tells how far into that specific segment the requested address is.

2.2 Positive vs. Negative Growth

- **Positive Growth (Code and Heap):** Grow from lower offsets to higher offsets.

$$\text{Physical Address (PA)} = \text{Segment Base} + \text{Offset}$$

$$\text{Bounds Check: Offset} < \text{Segment Size}$$

- **Negative Growth (Stack):** Grows from higher offsets to lower offsets.

$$\text{Negative Offset} = \text{Offset} - \text{Max Segment Size}$$

$$\text{Physical Address (PA)} = \text{Segment Base} + \text{Negative Offset}$$

$$\text{Bounds Check: } |\text{Negative Offset}| \leq \text{Segment Size}$$

Note: Max Segment Size is equal to 2^{offset_bits}.

3 Coursework Practice Problems Solved

Here are the complete, detailed solutions to the practice problems shared in your coursework slides.

3.1 Problem 1: Heap Management (Free List Allocation)

Question: A system uses a Free List to manage a heap. Initially, the free list contains blocks of the following sizes in this order:

$$[\text{Node A} : 15 \text{ bytes}] \rightarrow [\text{Node B} : 25 \text{ bytes}] \rightarrow [\text{Node C} : 20 \text{ bytes}]$$

The header for each allocated block requires an additional 5 bytes (not included in the requested size). If a user calls `malloc(12)`:

1. Which node is selected if the First Fit strategy is used?
2. Which node is selected if the Best Fit strategy is used?
3. Show the updated free list when the allocation is done for the Best Fit case.

Step-by-Step Solution:

- **Total Size Needed:** The allocation requires the requested size plus the header size:

$$\text{Total Space Needed} = 12 \text{ bytes (data)} + 5 \text{ bytes (header)} = 17 \text{ bytes}$$

- **First Fit:** It scans the list from left to right and selects the first block that is ≥ 17 bytes.
  - Node A (15 bytes): Too small ($15 < 17$).
  - Node B (25 bytes): Large enough ($25 \geq 17$).

  Answer: Node B is selected.

- **Best Fit:** It scans the entire list to find the block closest in size to 17 bytes (the smallest block that is ≥ 17 bytes).
  - Eligible blocks are Node B (25 bytes) and Node C (20 bytes).
  - Node C (20 bytes) is closer to 17 than Node B.

  Answer: Node C is selected.

- **Updated Free List (Best Fit):** We allocate 17 bytes from Node C.

$$\text{Remaining space in Node C} = 20 \text{ bytes} - 17 \text{ bytes} = 3 \text{ bytes}$$

  Answer: [Node A : 15 bytes] $\rightarrow$ [Node B : 25 bytes] $\rightarrow$ [Node C : 3 bytes].

3.2 Problem 2: Base and Bounds Translation

Question: A process has a Base Register set to physical address `0x4000` (16 KB) and a Bounds Register set to `0x0800` (2 KB). Translate the following Virtual Addresses (VA) to Physical Addresses (PA). If an address is invalid, state "Segmentation Fault".

1. VA: `0x0100`
2. VA: `0x0800`

3. VA: `0x0050`

Step-by-Step Solution:

- **Rule:** The hardware check is strictly: VA < Bounds. Here, Bounds = `0x0800`.
- **VA:** `0x0100`
  - Check: `0x0100` < `0x0800` (True).
  - Translate: PA = Base + VA = `0x4000` + `0x0100` = `0x4100`.
- **VA:** `0x0800`
  - Check: `0x0800` < `0x0800` (False).
  - **Answer: Segmentation Fault**.
- **VA:** `0x0050`
  - Check: `0x0050` < `0x0800` (True).
  - Translate: PA = Base + VA = `0x4000` + `0x0050` = `0x4050`.

3.3 Problem 3: Memory Compaction Difficulty

Question: Explain why Memory Compaction is extremely difficult to implement using Static Relocation but trivial to implement using Dynamic Relocation.

Step-by-Step Solution:

- **Static Relocation:** During program loading, the OS permanently rewrites code and data addresses to point directly to physical locations. If the OS decides to move a running process to a different physical RAM location (for compaction), it would have to find and rewrite every pointer/address within that process. Since the OS cannot reliably distinguish between a raw integer (e.g., 1000) and a memory pointer (address 1000), this rewrite is practically impossible.
- **Dynamic Relocation:** The running program is entirely oblivious to its physical location; it only operates with virtual addresses starting from 0. To compact, the OS simply copies the process's physical memory contents to a new location and updates the process's **Base register** in the MMU. No instructions inside the program are changed, making compaction simple, fast, and safe.

3.4 Problem 4: 14-Bit Segmented Translation

Question: A system uses a 14-bit Virtual Address. The top 2 bits select a segment, and the lower 12 bits represent the offset. The segment identifiers are: `00`=Code, `01`=Heap, `10`=Stack. Segment descriptors:

- Code Segment (`00`): Base = `0x2000`, Size = 8 KB, Growth = Positive
- Heap Segment (`01`): Base = `0x8000`, Size = 6 KB, Growth = Positive
- Stack Segment (`10`): Base = `0xF000`, Size = 4 KB, Growth = Negative

Translate: (1) `VA1 = 0x1010`, (2) `VA2 = 0x17F3`.

Step-by-Step Solution:

- **Analyze VA1 =** `0x1010`**:**

1. Convert to 14-bit binary: `0x1010` → **01 0000 0001 0000**.
2. Segment Selector: Top 2 bits are `01` → **Heap Segment**.
3. Offset: Remaining 12 bits are `0x010` (16 bytes).
4. Bounds Check: 16 bytes < Heap Size (6 KB = 6144 bytes) is valid.
5. Translate: PA = Base + Offset = `0x8000` + `0x010` = `0x8010`.

- **Analyze VA2 = `0x17F3`:**

1. Convert to 14-bit binary: `0x17F3` → **01 0111 1111 0011**.
2. Segment Selector: Top 2 bits are `01` → **Heap Segment**.
3. Offset: Remaining 12 bits are `0x7F3` (2035 bytes).
4. Bounds Check: 2035 bytes < Heap Size (6 KB = 6144 bytes) is valid.
5. Translate: PA = Base + Offset = `0x8000` + `0x7F3` = `0x87F3`.

3.5 Problem 5: Assembly Instruction Access Tracking

Question: Consider the assembly instructions located at virtual addresses starting from `0x2000`:

```
0x2000: mov (%rbx), %eax
0x2003: add $0x4, %eax
0x2007: mov %eax, (%rcx)
```

Calculate the total memory accesses, breaking them down by type (Instruction Fetches / Data Loads / Data Stores).

Step-by-Step Solution:

1. Every instruction must be retrieved from memory before running. This is an **Instruction Fetch**.
2. **Instruction 1 (`mov (%rbx), %eax`):**
   - Fetch instruction from `0x2000` (1 Instruction Fetch).
   - Reads data from the virtual address stored in `%rbx` into register `%eax` (1 Data Load).
3. **Instruction 2 (`add $0x4, %eax`):**
   - Fetch instruction from `0x2003` (1 Instruction Fetch).
   - Operates purely on registers inside the CPU (0 data memory accesses).
4. **Instruction 3 (`mov %eax, (%rcx)`):**
   - Fetch instruction from `0x2007` (1 Instruction Fetch).
   - Writes data in register `%eax` to the virtual address in `%rcx` (1 Data Store).

Answer Summary:

- **Instruction Fetches:** 3
- **Data Loads:** 1
- **Data Stores:** 1
- **Total Memory Accesses:** 5

3.6 Problem 6: Multi-Segment Design & Translation

Question: We want to support 5 distinct segments (Code, Data, Heap, Stack 1, Stack 2) using a 14-bit Virtual Address.

1. How many bits are needed for the Segment Selector? What is the Max Segment Size?
2. Translate: (i) VA = `0x0100`, (ii) VA = `0x1050` using:
   - Seg `000` (Code): Base = `0x2000`, Size = 2 KiB (`0x0800`)
   - Seg `010` (Heap): Base = `0x5000`, Size = 1 KiB (`0x0400`)

Step-by-Step Solution:

1. **Selector Bits:** To uniquely identify 5 states, we need 3 bits (since $2^2 = 4 < 5$, and $2^3 = 8 \geq 5$).
2. **Max Segment Size:** Out of 14 virtual bits, 3 bits are used for the selector. This leaves $14 - 3 = 11$ bits for the offset.

$$\text{Max Segment Size} = 2^{11} \text{ bytes} = 2048 \text{ bytes (2 KiB)}$$

3. **Translation Calculations:**
   - **VA = `0x0100`:**
     - 14-bit binary: **000 0010 0000 0000**.
     - Selector: `000` → **Code**.
     - Offset: `0x100` (256 bytes).
     - Check: $256 < 2048$ (Size of Code is 2 KiB) → Valid.
     - PA = Base + Offset = `0x2000` + `0x0100` = `0x2100`.
   - **VA = `0x1050`:**
     - 14-bit binary: **010 0000 0101 0000**.
     - Selector: `010` → **Heap**.
     - Offset: `0x050` (80 bytes).
     - Check: $80 < 1024$ (Size of Heap is 1 KiB) → Valid.
     - PA = Base + Offset = `0x5000` + `0x050` = `0x5050`.

3.7 Problem 7: Stack-Frame Lifetimes and Undefined Behavior

Question: Based on the concept of "Stack for Procedure Invocation Frames," explain why the value printed at Call C might be different from Call A, or why the program might exhibit undefined behavior, even though the pointer p has not changed in the C code provided.

Step-by-Step Solution:

1. **Allocation on Stack:** When `get_ptr()` executes, a stack frame is allocated on the runtime stack to store its local variable $x = 100$.
2. **Deallocation:** When the function returns, its stack frame is popped off and marked as free. However, the raw data value 100 physically remains intact inside that memory location as **lingering data**.
3. **Dangling Pointer:** The pointer p holds the address of x, which is now in an unallocated region of the stack. It is a **dangling pointer**.

4. **Call A vs. Call C:**

   - During **Call A**, the value 100 is successfully printed because no other functions have run yet, so the stack memory has not been overwritten.
   - When `int y = 50` is initialized, and other local variables are declared, a new stack frame is created. Because the stack grows downward, the CPU overwrites the previously deallocated space with the new local variable values.
   - By the time **Call C** executes, the memory location pointed to by p has been overwritten by the value of y (or other garbage data). Thus, Call C prints a different value, leading to undefined behavior.

4 Similar "Clone" Practice Problems (For Exam Mastery)

Solve these similar problems to guarantee you can answer anything on the exam tomorrow.

4.1 Clone Problem 1: Free List Heap Allocation

Question: A free list initially contains blocks in this order:

$$[\text{Node X : 12 bytes}] \rightarrow [\text{Node Y : 32 bytes}] \rightarrow [\text{Node Z : 20 bytes}]$$

Each allocated block requires an additional 4-byte header. If a user calls `malloc(16)`:

1. Which node is selected if First Fit is used?
2. Which node is selected if Best Fit is used?
3. What is the updated free list after Best Fit allocation?

Detailed Solution:

- Space needed = 16 bytes (data) + 4 bytes (header) = 20 bytes.
- **First Fit:** Scans left-to-right.
  - Node X (12B): Too small ($12 < 20$).
  - Node Y (32B): Large enough ($32 \geq 20$).

  Answer: Node Y is selected.
- **Best Fit:** Scans list to find closest size.
  - Eligible blocks are Node Y (32B) and Node Z (20B).
  - Node Z (20B) is a perfect fit.

  Answer: Node Z is selected.
- **Updated Free List (Best Fit):** Node Z is reduced by 20 bytes, leaving 0 bytes (removed from list). **Answer:** [Node X : 12 bytes] → [Node Y : 32 bytes].

4.2 Clone Problem 2: Dynamic Relocation Translation

Question: A process has a Base Register set to physical address `0x5000` and a Bounds Register set to `0x0400` (1 KB). Translate these VAs: (1) `0x0250`, (2) `0x0400`, (3) `0x0450`. **Detailed Solution:**

- Bounds Check condition: VA < `0x0400`.
- **VA `0x0250`:** 0x0250 < 0x0400 (True) → PA = 0x5000 + 0x0250 = `0x5250`.
- **VA `0x0400`:** 0x0400 < 0x0400 (False) → **Segmentation Fault**.
- **VA `0x0450`:** 0x0450 < 0x0400 (False) → **Segmentation Fault**.

4.3 Clone Problem 4: 14-Bit Segmented Translation with Negative Stack Growth

Question: A system uses a 14-bit Virtual Address (top 2 bits = selector, lower 12 bits = offset). Descriptors:

- Code Segment (`00`): Base = `0x1000`, Size = 2 KB, Growth = Positive
- Stack Segment (`10`): Base = `0xC000`, Size = 2 KB, Growth = Negative

Translate: (1) `VA1 = 0x0200`, (2) `VA2 = 0x2E00`. **Detailed Solution:**

- **Analyze VA1 =** `0x0200`**:**
  1. 14-bit binary: **00 0010 0000 0000**.
  2. Selector: `00` → **Code**.
  3. Offset: `0x200` (512 bytes).
  4. Check: $512 < 2048$ (Size is 2 KB) → Valid.
  5. PA = Base + Offset = `0x1000` + `0x0200` = `0x1200`.
- **Analyze VA2 =** `0x2E00`**:**
  1. 14-bit binary: **10 1110 0000 0000**.
  2. Selector: `10` → **Stack** (Negative Growth!).
  3. Offset: `0xE00` (3584 bytes).
  4. Max Segment Size: Since offset is 12 bits, Max Size = $2^{12} = 4096$ bytes.
  5. Negative Offset = Offset - Max Segment Size = $3584 - 4096 = \mathbf{-512}$ bytes.
  6. Bounds Check: $|-512| = 512$ bytes $\leq$ Stack Size (2 KB = 2048 bytes) is valid.
  7. PA = Base + Negative Offset = `0xC000` + (-512) = `0xBE00`.

4.4 Clone Problem 5: Assembly Access Tracking

Question: Calculate the memory accesses for these instructions running sequentially starting at `0x3000`:

```
0x3000: mov (%rax), %ebx
0x3003: mov (%rcx), %edx
0x3006: add %ebx, %edx
0x3008: mov %edx, (%rsi)
```

Detailed Solution:

- **Instruction Fetches:** 4 (One fetch for each of the 4 instructions).
- **Data Loads:** 2 (Instruction 1 loads from address in `%rax`, Instruction 2 loads from address in `%rcx`).
- **Data Stores:** 1 (Instruction 4 stores value into address in `%rsi`).
- **Total Accesses:** 4 Fetches + 2 Loads + 1 Store = **7** Accesses.

5 Paging and Page Tables (Advanced Mastery)

This section prepares you for tomorrow's most challenging mathematical problems.

5.1 Paging Foundations

- **Virtual Page Number (VPN):** The upper bits of a virtual address that select which entry to look up in the Page Table.
- **Offset:** The lower bits of the address that remain identical during physical translation. If page size is 2^n bytes, offset is n bits.
- **Page Frame Number (PFN):** The physical page number retrieved from the page table.

5.2 Flat Page Table Calculations

- **Example 1:** 32-bit Virtual Address, 4 KiB (2^{12} B) Page Size, 4-byte Page Table Entry (PTE).

$$\text{Entries} = \frac{2^{32}}{2^{12}} = 2^{20}$$

$$\text{Flat Page Table Size} = 2^{20} \times 4 \text{ B} = 2^{22} \text{ B} = \mathbf{4} \text{ MiB}$$

- **Example 2:** 32-bit Virtual Address, 8 KiB (2^{13} B) Page Size, 8-byte PTE.

$$\text{Entries} = \frac{2^{32}}{2^{13}} = 2^{19}$$

$$\text{Flat Page Table Size} = 2^{19} \times 8 \text{ B} = 2^{22} \text{ B} = \mathbf{4} \text{ MiB}$$

5.3 Multi-Level Page Tables

Multi-level page tables break a linear page table into page-sized pieces to save space.

The 32-bit Two-Level Table

For a 32-bit VA, 4 KiB Page, and 4-byte PTE:

- **Offset bits:** 12 bits (since $2^{12} = 4$ KiB).
- **PTEs per page:** $\frac{4 \text{ KiB}}{4 \text{ bytes}} = 1024 = 2^{10}$. This requires **10 bits** for the Page Table Index.
- **Page Directory Index (First-level):** Remaining bits: $32 - 12 - 10 = \mathbf{10}$ bits.
- **Address Format:** `[PD Index (10 bits) | PT Index (10 bits) | Offset (12 bits)]`

5.4 Translation Lookaside Buffer (TLB) & AMAT

Average Memory Access Time (AMAT) is a high-yield exam topic.

$$\text{AMAT} = \text{Hit Rate} \times T_{\text{Hit}} + \text{Miss Rate} \times T_{\text{Miss}}$$

Example Problem: A system has a 2-level page table, a memory access time of 100 ns, a TLB lookup time of 5 ns, and a 95% hit rate.

- **On a TLB hit:** MMU looks up TLB (5 ns) and then accesses physical data in memory (100 ns).

$$T_{\text{Hit}} = 5 + 100 = 105 \text{ ns}$$

- **On a TLB miss:** MMU checks TLB (5 ns), walks the 2-level page table in memory ($2 \times 100 = 200$ ns), and then accesses physical data (100 ns).

$$T_{\text{Miss}} = 5 + 3 \times 100 = 305 \text{ ns}$$

- **AMAT Calculation:**

$$\text{AMAT} = (0.95 \times 105) + (0.05 \times 305) = 99.75 + 15.25 = \mathbf{115} \text{ ns}$$

5.5 Page Replacement Policies

FIFO (First-In-First-Out), LRU (Least Recently Used), and Optimal are the main three algorithms tested.

Table 1: Worked example trace with 3 physical frames (initially empty) for reference string: 3, 1, 4, 1, 5, 9, 2.

Reference	**3**	**1**	**4**	**1**	**5**	**9**	**2**
FIFO Frames	[3, -, -]	[3, 1, -]	[3, 1, 4]	[3, 1, 4]	[5, 1, 4]	[5, 9, 4]	[5, 9, 2]
Page Fault?	Fault	Fault	Fault	Hit	Fault (evicts 3)	Fault (evicts 1)	Fault (evicts 4)
LRU Frames	[3, -, -]	[3, 1, -]	[3, 1, 4]	[3, 1, 4]	[5, 1, 4]	[5, 1, 9]	[2, 1, 9]
Page Fault?	Fault	Fault	Fault	Hit	Fault (evicts 3)	Fault (evicts 4)	Fault (evicts 5)
Optimal	[3, -, -]	[3, 1, -]	[3, 1, 4]	[3, 1, 4]	[5, 1, 4]	[5, 1, 9]	[5, 1, 2]
Page Fault?	Fault	Fault	Fault	Hit	Fault (evicts 3)	Fault (evicts 4)	Fault (evicts 9)